

Quadcopter Equations of Motion

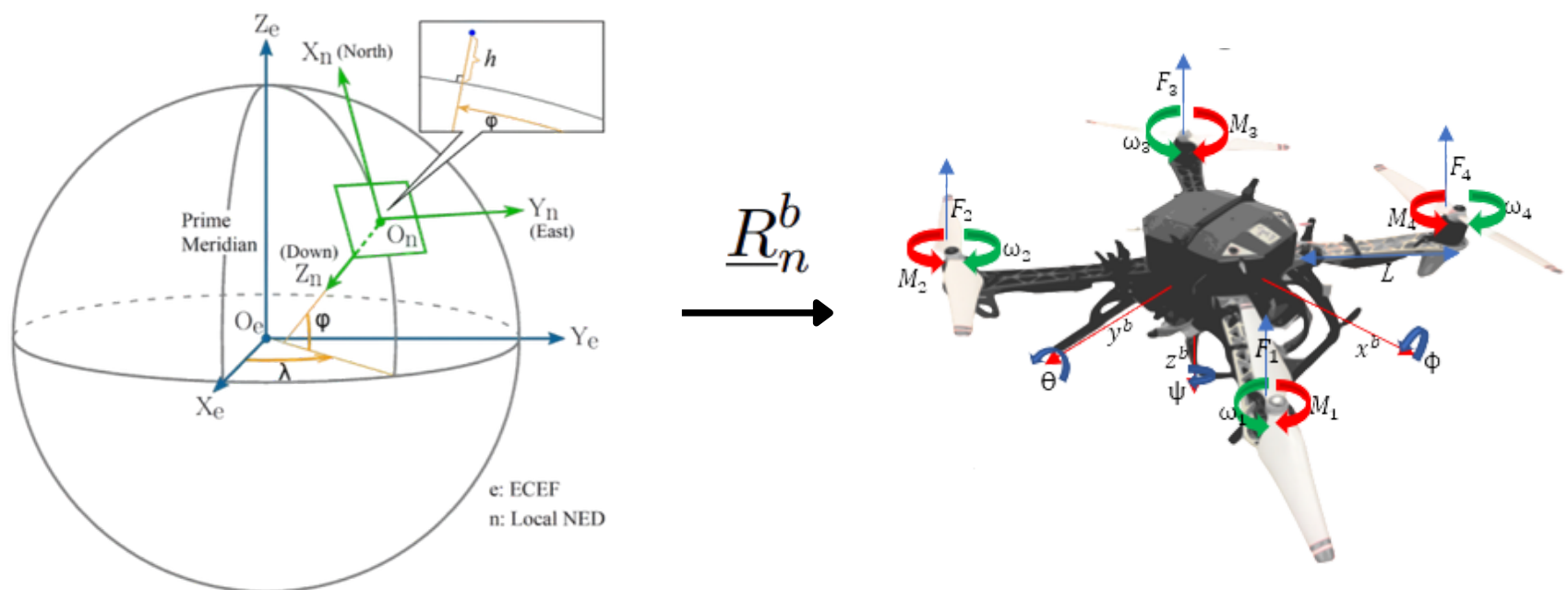
The math
that keeps a
drone in the air



Reference Frames

NED & Body Frame

A reference frame is the "point of view" you use to describe the position, velocity, and orientation of an object.



NED frame (navigation frame)
is the fixed world reference:
It tells where the drone is
moving relative to the Earth.

x_n = North

y_n = East

z_n = Down

**Body frame is attached to
the drone itself:**
It moves and rotates with
the drone (quadcopter).

x^b = forward

y^b = right

z^b = down

Orientation

Euler Angles & Rotation Matrix

How does a drone know where it's pointing in 3D space?

$$\begin{aligned}\underline{R}_n^b &= \underline{R}_x(\phi)\underline{R}_y(\theta)\underline{R}_z(\psi) \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & \sin\phi \\ 0 & -\sin\phi & \cos\phi \end{bmatrix} \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ \underline{R}_n^b &= \begin{bmatrix} c\theta c\psi & c\theta s\psi & -s\theta \\ s\phi s\theta c\psi - c\phi s\psi & s\phi s\theta s\psi + c\phi c\psi & s\phi c\theta \\ c\phi s\theta c\psi + s\phi s\psi & c\phi s\theta s\psi - s\phi c\psi & c\phi c\theta \end{bmatrix}\end{aligned}$$

$c = \cos, s = \sin, \phi = \text{roll}, \theta = \text{pitch}, \psi = \text{yaw}$

This 3×3 rotation matrix performs a rotation from the navigation frame (NED), n , to the body-fixed frame, b , of the quadcopter. It encodes all three Euler angles into one compact rotation operator.

$\underline{R}_b^n$ the transpose of $\underline{R}_n^b$, does the reverse rotation.

Kinematics

Kinematic Model

Linking body-frame motion to navigation-frame (NED) motion

$$\underline{v} = \underline{R}_b^n \cdot \underline{v}_b \qquad \underline{w} = \underline{T}_b^n \cdot \underline{w}_b$$

$\underline{v}_b = (u, v, w)^T \in \mathbb{R}^3 \leftarrow$ linear velocity in the body frame

$\underline{v} = (\dot{x}, \dot{y}, \dot{z})^T \in \mathbb{R}^3 \leftarrow$ linear velocity in the NED frame

$\underline{w}_b = (p, q, r)^T \in \mathbb{R}^3 \leftarrow$ angular velocity in the body frame

$\underline{w} = (\dot{\phi}, \dot{\theta}, \dot{\psi})^T \in \mathbb{R}^3 \leftarrow$ Euler angle rates

$\underline{T}_b^n$ the Euler Kinematic Matrix, maps body-frame angular rates to Euler angle rates

For small-angle motion: $\cos(\theta) \approx 1, \sin(\theta) \approx \theta$

$$(p, q, r)^T \approx (\dot{\phi}, \dot{\theta}, \dot{\psi})^T, \quad (u, v, w)^T \approx (\dot{x}, \dot{y}, \dot{z})^T$$

The drone's IMU measures velocities and angular rates in its own body frame. The kinematic model uses R to map linear velocity and T to map angular rates into the global NED frame.

Newton's EOM for Quadcopters

What actually moves a drone through 3D space?

$$\underline{f}_b = m (\underline{\dot{v}}_b + \underline{\omega}_b \times \underline{v}_b) \leftarrow \text{total external force expressed in the body frame}$$

$$\underline{\dot{v}} = \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = -\frac{T}{m} \begin{bmatrix} c\phi s\theta c\psi + s\phi s\psi \\ c\phi s\theta s\psi - s\phi c\psi \\ c\phi c\theta \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} \leftarrow \text{NED frame}$$

$\underline{\dot{v}}$ $\leftarrow$ linear acceleration in NED frame

m $\leftarrow$ total drone mass (kg)

$T = F_1 + F_2 + F_3 + F_4 \leftarrow$ total thrust from all motors

$g = 9.81 \text{ m/s}^2 \leftarrow$ gravity always acts on +Z (down)

Total thrust T is generated by 4 rotors.
The rotation matrix projects it into X, Y, Z accelerations.
Gravity always acts downward (+Z in NED).

Euler's EOM for Quadcopters

How does the drone tilt, roll, and spin?

$\underline{M}_b = (\underline{J} \cdot \underline{\dot{w}}_b + \underline{w}_b \times (\underline{J} \cdot \underline{w}_b)) \leftarrow$ total torque in the body frame

$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} M_\phi J_{xx}^{-1} \\ M_\theta J_{yy}^{-1} \\ M_\psi J_{zz}^{-1} \end{bmatrix} - \begin{bmatrix} qr(J_{zz} - J_{yy})/J_{xx} \\ pr(J_{xx} - J_{zz})/J_{yy} \\ pq(J_{yy} - J_{xx})/J_{zz} \end{bmatrix}$$

$(p, q, r)^T \approx (\dot{\phi}, \dot{\theta}, \dot{\psi})^T \leftarrow$ small-angle approximation

$M_\phi = L(-F1 - F2 + F3 + F4) \leftarrow$ roll torque (L = arm length)

$M_\theta = L(F1 - F2 - F3 + F4) \leftarrow$ pitch torque

$M_\psi = kd(F1 - F2 + F3 - F4) \leftarrow$ yaw torque (kd = drag coeff)

Inertia $J_{xx}/J_{yy}/J_{zz}$ resists angular acceleration just as mass resists linear acceleration. The cross-product terms (gyroscopic coupling) appear when the drone rotates on multiple axes simultaneously.

Full 6-DOF Model

The Complete Quadcopter Model

State vector: $[x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}]$ — 12 states

01 Translational EOM

Controls x, y, z position. Driven by total thrust T .

02 Rotational EOM

Controls ϕ, θ, ψ attitude. Driven by differential torques M .

03 Kinematic model

Maps body-frame velocities (u, v, w) & (p, q, r) to world frame.

04 Next step

Linearize around hover → build PID controller.

Linearization

$$\Delta \dot{\underline{x}} = f(\underline{x}_0, \underline{u}_0, t) + \frac{\partial f(\underline{x}, \underline{u}, t)}{\partial \underline{x}} \Big|_{\underline{x}_0, \underline{u}_0} \Delta \underline{x} + \frac{\partial f(\underline{x}, \underline{u}, t)}{\partial \underline{u}} \Big|_{\underline{x}_0, \underline{u}_0} \Delta \underline{u} + \dots$$

$$\underline{x} = (\phi, \dot{\phi}, \theta, \dot{\theta}, \psi, \dot{\psi}, x, \dot{x}, y, \dot{y}, z, \dot{z})^T \in \mathbb{R}^{12}$$

$$\underline{u} = (T, M_\phi, M_\theta, M_\psi)^T \in \mathbb{R}^4$$

$$\underline{x}_0 = (0, 0, 0, 0, \psi_0, 0, x_0, 0, y_0, 0, z_0, 0)^T \in \mathbb{R}^{12}$$

$$\underline{u}_0 = (mg, 0, 0, 0)^T \in \mathbb{R}^4$$

$$\underline{A} = \frac{\partial f(\underline{x}, \underline{u}, t)}{\partial \underline{x}} \Big|_{\underline{x}_0, \underline{u}_0}$$

$$\underline{B} = \frac{\partial f(\underline{x}, \underline{u}, t)}{\partial \underline{u}} \Big|_{\underline{x}_0, \underline{u}_0}$$

$$\left\{ \begin{array}{l} \dot{\phi} = p \\ \ddot{\phi} = M_\phi J_{xx}^{-1} \\ \dot{\theta} = q \\ \ddot{\theta} = M_\theta J_{yy}^{-1} \\ \dot{\psi} = r \\ \ddot{\psi} = M_\psi J_{zz}^{-1} \\ \dot{x} = u \\ \ddot{x} = -g\phi s\psi_0 - g\theta c\psi_0 \\ \dot{y} = v \\ \ddot{y} = g\phi c\psi_0 - g\theta s\psi_0 \\ \dot{z} = w \\ \ddot{z} = -T/m \end{array} \right.$$

Input forces and moments

- X - configuration and NED coordinate.
- $T = F_1 + F_2 + F_3 + F_4$
- $M_\phi = l(-F_1 - F_2 + F_3 + F_4)$
- $M_\theta = l(F_1 - F_2 - F_3 + F_4)$
- $M_\psi = (M_1 - M_2 + M_3 - M_4)$

Motor mixing algorithm

$$\begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{bmatrix} = \begin{bmatrix} 0.25 & -0.25/l & 0.25/l & 0.25c/b \\ 0.25 & -0.25/l & -0.25/l & -0.25c/b \\ 0.25 & 0.25/l & -0.25/l & 0.25c/b \\ 0.25 & 0.25/l & 0.25/l & -0.25c/b \end{bmatrix} \begin{bmatrix} T \\ M_\phi \\ M_\theta \\ M_\psi \end{bmatrix}$$

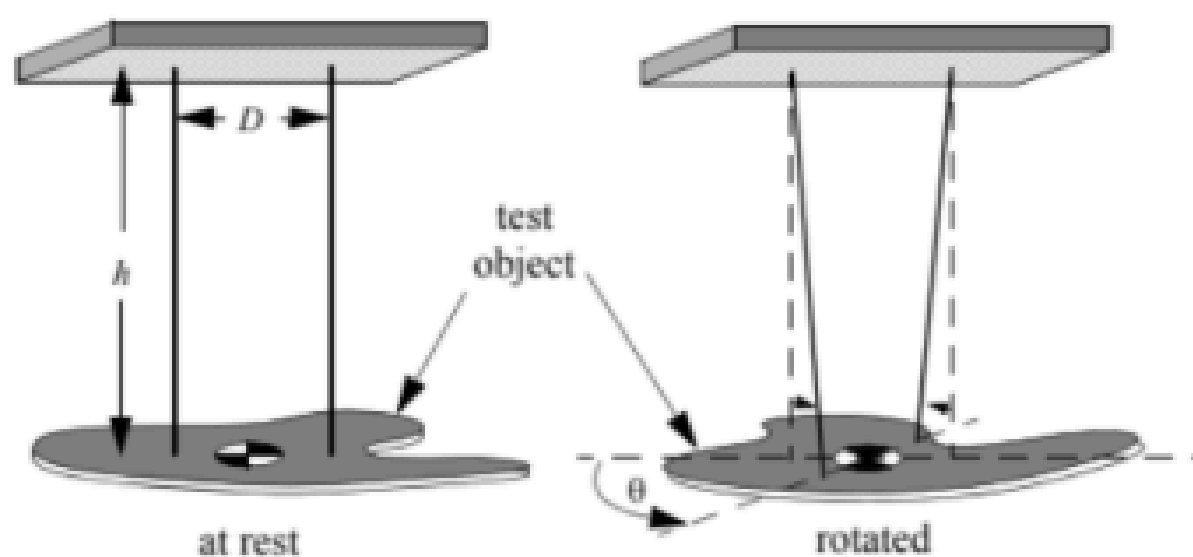
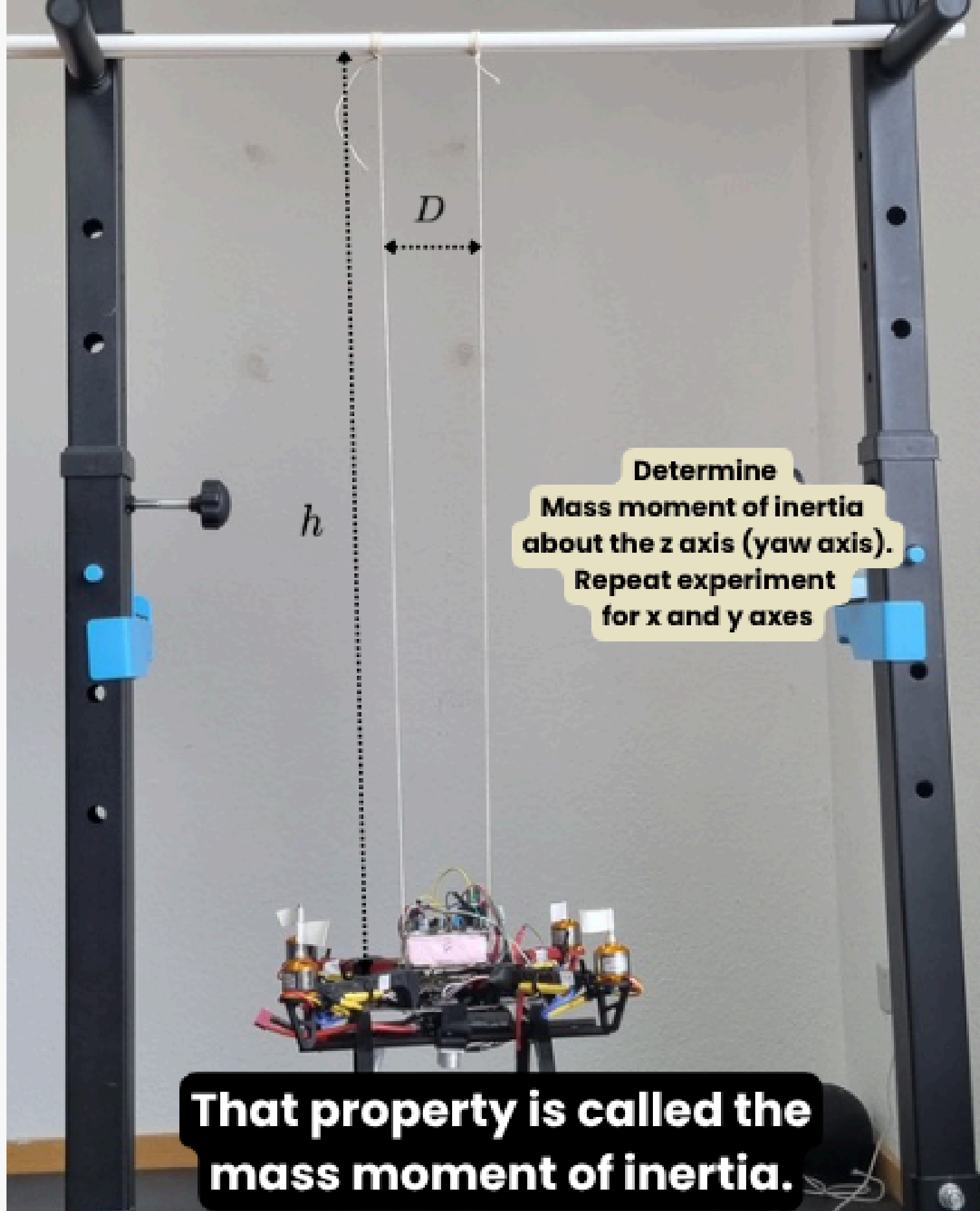


Figure 1. Bifilar pendulum diagrams

source: Optimized Measurements of UAV Mass Moment of Inertia with a Bifilar Pendulum, Matt R. Jordin^a Eric R. Mueller^b

@huobuilds

Bifilar Pendulum Experiment

A bifilar pendulum uses two strings to measure how much an object resists twisting about a vertical axis.

release it and count the number of oscillations.

6x Speed

9

00:41.98

Mass Moment of Inertia

$$J = \frac{mgD^2T^2}{16\pi^2h}$$

$$T = \frac{t_{total}}{n} = \frac{48}{10} = 4.8 \text{ sec}$$

period of one oscillation

Parameter	Symbol	Value	Unit
Mass of quadcopter	m	0.89	kg
String length/height	h	0.77	m
String separation	D	0.084	m
Number of oscillations counted	n	10	—
Total time for 10 oscillations	t_{total}	48	s
Gravitational acceleration	g	9.81	m/s ²

$$J_{zz} = \frac{0.89 \times 9.81 \times (0.084)^2 \times (4.8)^2}{16 \times \pi^2 \times 0.77} = 0.01167 \text{ kg} \cdot \text{m}^2$$

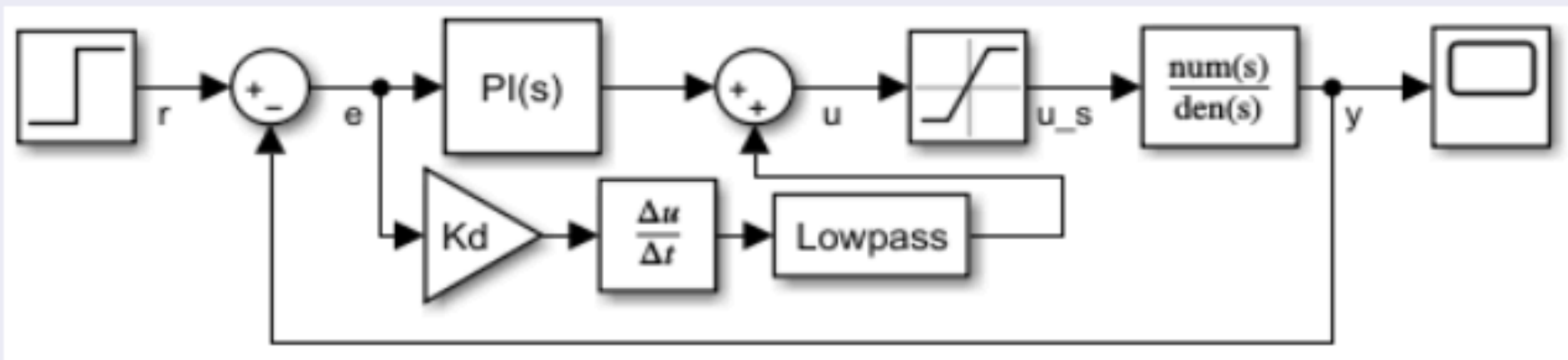
$$J_{zz} = 0.01167 \text{ kg} \cdot \text{m}^2$$

$$F_{cl}(s) = \frac{s^2(k_p + k_d\omega_c)J_{zz}^{-1} + s(k_i + k_p\omega_c)J_{zz}^{-1} + k_i\omega_cJ_{zz}^{-1}}{s^3(s + \omega_c) + s^2(k_p + k_d\omega_c)J_{zz}^{-1} + s(k_i + k_p\omega_c)J_{zz}^{-1} + k_i\omega_cJ_{zz}^{-1}}$$

PID controller + Drone Closed loop transfer function

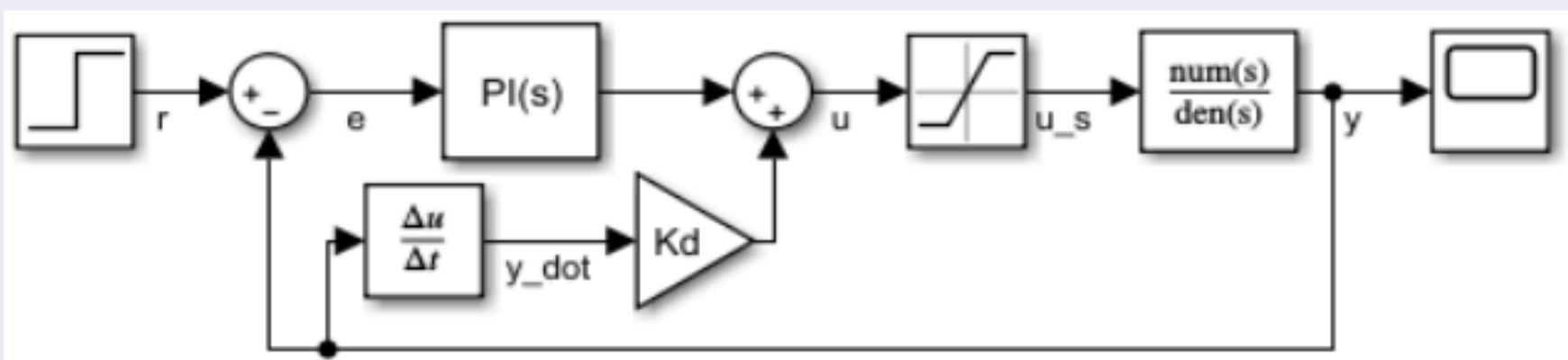
**Now you can model and tune
your drone PID controller.**

PID: derivative kickback and noise

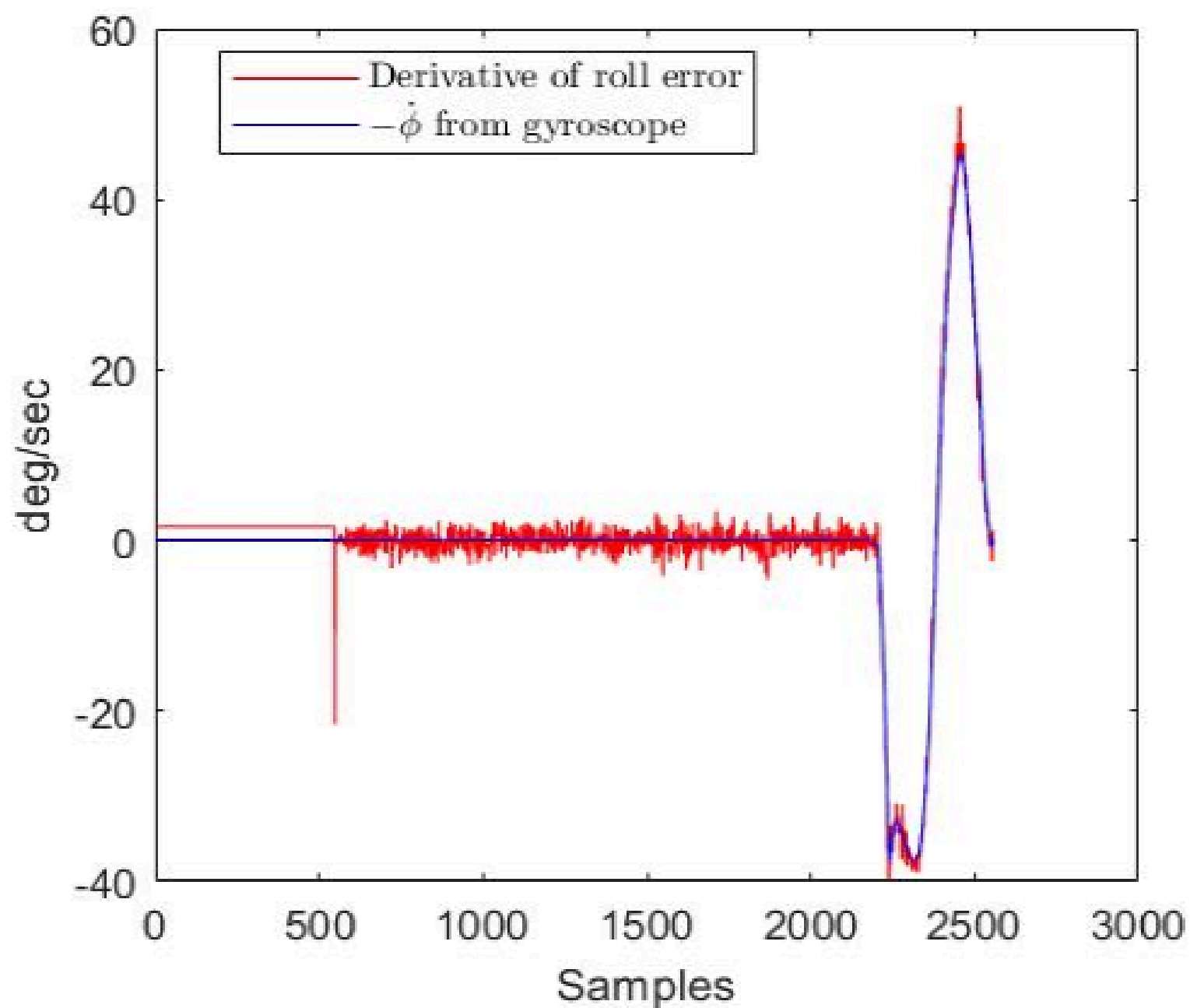


$$u_k = K_p e_k + K_i T \sum_{j=0}^{k-1} e_j + \frac{K_d}{T} (e_k - e_{k-1})$$

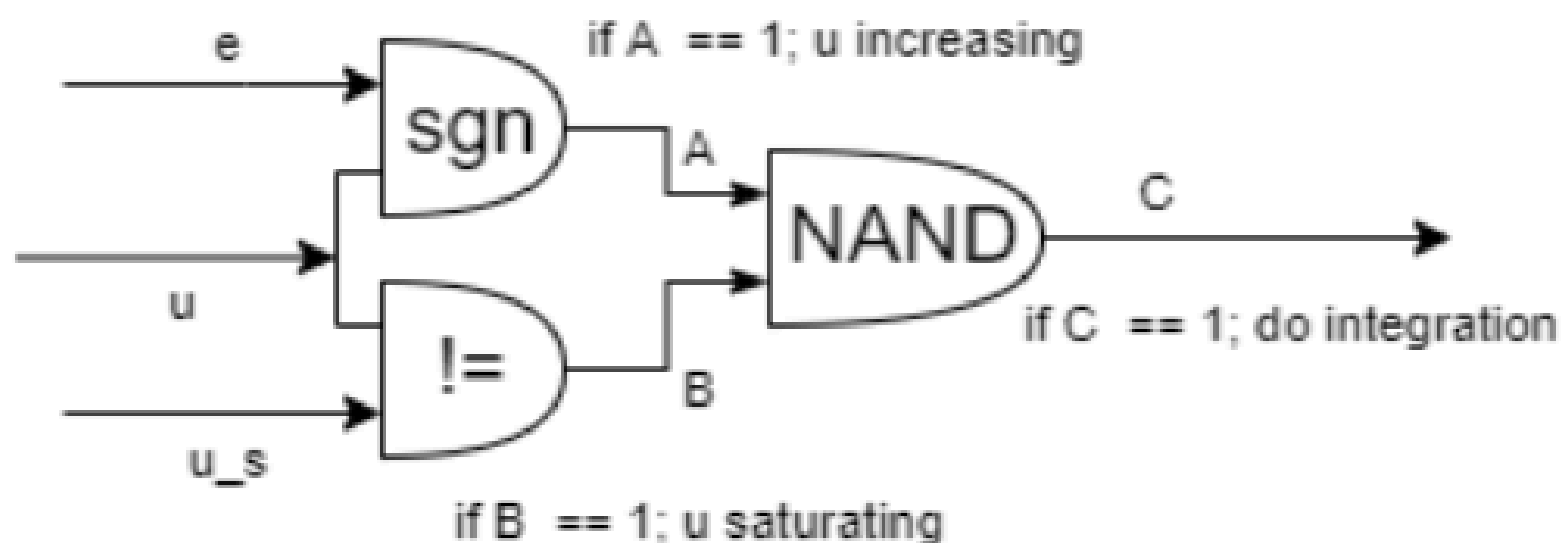
2DOF PID: No derivative kickback and Lowpass filter not needed



$$u_k = K_p e_k + K_i T \sum_{j=0}^{k-1} e_j - K_d \dot{\phi}_k$$



integral anti-windup clamping



A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

Want the full drone build-from-scratch series?

Follow @huobuilds
for real engineering
breakdowns



Save this for later